

Linked Lists — Question Bank with Answers

Below is a **50-question exam-oriented question bank** covering the topics you specified:

- Singly Linked Lists
 - Memory representation
 - Traversal
 - Searching
 - Insertion
 - Deletion
 - Linked Stack and Queue
 - Header nodes
 - Doubly Linked Lists
 - Circular Linked Lists
 - Algorithms and complexity analysis
-

Part A — 30 Questions × 2 Marks

Q1. What is a linked list?

Answer:

A linked list is a dynamic linear data structure consisting of a collection of nodes. Each node contains **data** and one or more **links/pointers** connecting it to other nodes.

Q2. What are the two main fields of a node in a singly linked list?

Answer:

A singly linked-list node contains:

1. **Data field** — stores the element.
 2. **Next/link field** — stores the address/reference of the next node.
-

Q3. What is the purpose of the head pointer?

Answer:

The **head pointer** stores the address of the first node of a linked list. If the list is empty,

```
head = NULL .
```

Q4. What does the next field of the last node contain in a singly linked list?

Answer:

The next field of the last node contains **NULL**, indicating that there is no next node.

Q5. Give two advantages of linked lists over arrays.

Answer:

1. Linked lists can **grow or shrink dynamically**.
 2. Insertion and deletion can be performed without shifting other elements.
-

Q6. Give two disadvantages of linked lists.

Answer:

1. Extra memory is required for storing pointers.
 2. Direct/random access to an element is not possible; traversal is required.
-

Q7. What is traversal of a linked list?

Answer:

Traversal is the process of **visiting each node of a linked list sequentially**, starting from the first node and following the links until the end.

Complexity: $O(n)$

Q8. What is the time complexity of searching an element in a singly linked list?

Answer:

The worst-case time complexity is **$O(n)$** because the element may be located at the last node or may not exist.

Q9. What is the best-case complexity of searching a linked list?

Answer:

The best case is **$O(1)$** , when the required element is found at the first node.

Q10. What is the complexity of inserting a node at the beginning of a singly linked list?

Answer:

The complexity is **O(1)** because only the new node's link and the head pointer need to be modified.

Q11. Write the basic steps for inserting a node at the beginning.

Answer:

```
newNode->next = head  
head = newNode
```

The new node becomes the first node.

Q12. What is the complexity of deleting the first node?

Answer:

The complexity is **O(1)**.

Basic operation:

```
temp = head  
head = head->next  
delete temp
```

Q13. What is the complexity of inserting at the end of a singly linked list without a tail pointer?

Answer:

It is **O(n)** because the list must be traversed to find the last node.

Q14. How can insertion at the end be made O(1)?

Answer:

Maintain a **tail pointer** that directly points to the last node. Then insertion at the end can be performed in **O(1)**.

Q15. What is a linked stack?

Answer:

A linked stack is a stack implemented using linked-list nodes. The **top pointer** identifies the top element, and push/pop operations are normally performed at the beginning of the list.

Q16. What is the time complexity of push and pop in a linked stack?

Answer:

Operation	Complexity
Push	O(1)
Pop	O(1)

Q17. What is a linked queue?

Answer:

A linked queue is a queue implemented using linked-list nodes. It generally maintains **front** and **rear** pointers.

Q18. What are the complexities of enqueue and dequeue in a linked queue?

Answer:

Operation	Complexity
Enqueue	O(1)
Dequeue	O(1)

This assumes both front and rear pointers are maintained.

Q19. What is a header node?

Answer:

A header node is a **special node placed at the beginning of a linked list**. It may store

list-related information and is mainly used to simplify insertion and deletion operations.

Q20. What is a sentinel node?

Answer:

A sentinel node is a special dummy node used to simplify boundary cases in data-structure operations. A header node can act as a sentinel node.

Q21. What is a doubly linked list?

Answer:

A doubly linked list is a linked list in which each node contains three fields:

[Previous](#) | [Data](#) | [Next](#)

It supports traversal in both forward and backward directions.

Q22. State one advantage and one disadvantage of a doubly linked list.

Answer:

Advantage: It supports bidirectional traversal.

Disadvantage: It requires extra memory for the `previous` pointer.

Q23. What does the previous pointer of the first node contain?

Answer:

Normally, the `previous` pointer of the first node contains **NULL**.

Q24. What does the next pointer of the last node contain in a normal doubly linked list?

Answer:

The `next` pointer of the last node contains **NULL**.

Q25. What is a circular linked list?

Answer:

A circular linked list is a linked list in which the **last node points back to the first node** instead of containing NULL.

Q26. What is the main difference between a linear and circular linked list?

Answer:

Linear linked list:

```
Last → NULL
```

Circular linked list:

```
Last → First
```

Q27. Give one application of a circular linked list.

Answer:

Circular linked lists are useful in **round-robin scheduling**, where processes are repeatedly visited in a circular sequence.

Q28. What is the complexity of traversing a linked list containing n nodes?

Answer:

The time complexity is **$O(n)$** because every node must be visited once.

Q29. Why is binary search inefficient on a standard linked list?

Answer:

Binary search requires efficient random access to the middle element. A linked list does not provide direct indexing, so finding the middle element requires traversal.

Q30. Why does a linked list require extra memory compared with an array?

Answer:

Each linked-list node requires additional memory to store one or more **pointer/link fields** along with the data.

Part B — 20 Questions × 5 Marks

Q1. Explain the representation of a singly linked list in memory with an example.

Answer:

A singly linked list consists of nodes stored at different memory locations. Each node contains:

```
+-----+-----+
| Data  | Next  |
+-----+-----+
```

For example:

```
HEAD
  ↓
+---+-----+   +---+-----+   +---+-----+
| 10 | •----->| 20 | •----->| 30 | NULL |
+---+-----+   +---+-----+   +---+-----+
```

The nodes do not need to occupy consecutive memory locations.

For example:

```
Node 10 → Address 1000
Node 20 → Address 2500
Node 30 → Address 1800
```

The `next` field stores the address of the next node.

Complexity: Accessing a particular kth node requires $O(n)$ time in the worst case.

Q2. Explain the algorithm for traversing a singly linked list and analyze its complexity.

Answer:

Algorithm

```
TRAVERSE(head)
1. Set current = head
2. While current != NULL:
    a. Process current->data
    b. current = current->next
3. Stop
```

Example

10 → 20 → 30 → NULL

The algorithm visits:

10 → 20 → 30

Complexity

If there are n nodes, every node is visited once.

- **Time complexity:** $O(n)$
- **Space complexity:** $O(1)$, excluding output/storage used by processing.

Q3. Explain the searching operation in a singly linked list.

Answer:

Searching involves comparing the required value with the data stored in each node.

Algorithm

```
SEARCH(head, key)
1. current = head
2. While current != NULL:
    a. If current->data == key:
        return FOUND
    b. current = current->next
3. return NOT FOUND
```

For:

10 → 20 → 30 → 40 → NULL

Searching for 30 compares:

10 → 20 → 30

Complexity

- **Best case:** $O(1)$
- **Average case:** $O(n)$
- **Worst case:** $O(n)$
- **Space:** $O(1)$

Q4. Explain insertion at the beginning of a singly linked list.

Answer:

Suppose the original list is:

10 → 20 → 30 → NULL

To insert 5 :

Steps

1. Create a new node.
2. Store 5 in the data field.
3. Make the new node point to the current head.
4. Update head to the new node.

Algorithm

```
INSERT_BEGIN(head, value)
```

1. Create newNode
2. newNode->data = value
3. newNode->next = head
4. head = newNode
5. Return head

Result:

5 → 10 → 20 → 30 → NULL

Complexity

Time: $O(1)$

Space: $O(1)$ auxiliary space for the new node.

Q5. Explain insertion after a given node in a singly linked list.

Answer:

Suppose:

10 → 20 → 30 → NULL

We want to insert 25 after 20 .

Steps

1. Create a new node containing 25 .
2. Make `newNode->next` point to `20->next` .
3. Make `20->next` point to the new node.

Algorithm

INSERT_AFTER(P, value)

1. Create newNode
2. `newNode->data = value`
3. `newNode->next = P->next`
4. `P->next = newNode`

Result:

10 → 20 → 25 → 30 → NULL

Complexity

If node P is already known:

Time: $O(1)$

If we first need to search for P :

Time: $O(n)$

Q6. Explain deletion of the first node from a singly linked list.

Answer:

Consider:

10 → 20 → 30 → NULL

To delete 10 :

Algorithm

```
DELETE_BEGIN(head)
```

1. If head == NULL:
 Report UNDERFLOW
2. temp = head
3. head = head->next
4. Free/delete temp
5. Return head

After deletion:

20 → 30 → NULL

Complexity

- Time: **O(1)**
- Auxiliary space: **O(1)**

Q7. Explain deletion of a node from the middle of a singly linked list.

Answer:

Suppose:

10 → 20 → 30 → 40 → NULL

To delete 30 , we need the predecessor node 20 .

Algorithm

```
DELETE_NODE(P)
```

1. temp = P->next
2. P->next = temp->next
3. Delete/free temp

Result:

```
10 → 20 → 40 → NULL
```

If the predecessor is already known, deletion takes **$O(1)$** .

If we have to search for the node/predecessor first, the overall complexity is **$O(n)$** .

Q8. Explain how a stack is implemented using a linked list.

Answer:

A linked stack uses a pointer called `top`.



Push

Insert a node at the beginning:

```
newNode->next = top
top = newNode
```

Pop

Remove the first node:

```
temp = top
top = top->next
delete temp
```

Complexity

Operation	Time
Push	$O(1)$
Pop	$O(1)$
Peek	$O(1)$

A linked stack can dynamically grow until available memory is exhausted.

Q9. Explain implementation of a queue using a linked list.

Answer:

A linked queue normally maintains two pointers:

- `front`
- `rear`

Example:

```
FRONT                      REAR
  ↓                          ↓
10 → 20 → 30 → NULL
```

Enqueue

A new node is added at the rear.

```
rear->next = newNode
rear = newNode
```

Dequeue

The node at the front is removed.

```
temp = front
front = front->next
delete temp
```

If the queue becomes empty:

```
front = rear = NULL
```

Complexity

- Enqueue: **O(1)**
- Dequeue: **O(1)**

Q10. Explain header nodes and their advantages.

Answer:

A header node is a special node placed before the first actual data node.

Example:

```
HEADER → 10 → 20 → 30 → NULL
```

The header node may contain information such as the number of nodes or simply act as a sentinel.

Advantages

1. Simplifies insertion at the beginning.
2. Simplifies deletion operations.
3. Reduces special cases involving the first node.
4. Can store additional list information.
5. Provides a consistent starting point for traversal.

The header node is generally **not considered a normal data node**.

Q11. Explain the structure and advantages of a doubly linked list.

Answer:

A doubly linked list node contains three fields:

```
+-----+-----+-----+  
| Prev | Data | Next |  
+-----+-----+-----+
```

Example:

```
NULL ← 10 ⇌ 20 ⇌ 30 → NULL
```

Advantages

1. Supports forward traversal.
2. Supports backward traversal.
3. Deletion of a known node is easier.
4. Insertion before or after a known node can be performed efficiently.

Disadvantage

Each node requires an additional `prev` pointer, increasing memory usage.

Q12. Explain insertion into a doubly linked list.

Answer:

Suppose:

10 ⇌ 20 ⇌ 30

To insert 25 between 20 and 30 :

Steps

```
newNode->prev = 20
newNode->next = 30

20->next = newNode
30->prev = newNode
```

Result:

10 ⇌ 20 ⇌ 25 ⇌ 30

If the insertion position/node is already known:

Time complexity = $O(1)$

If searching for the position is required:

Overall complexity = $O(n)$

Q13. Explain deletion from a doubly linked list.

Answer:

Consider:

10 ⇌ 20 ⇌ 30

To delete 20 :

```
20->prev->next = 20->next
20->next->prev = 20->prev
delete 20
```

Result:

10 $\rightleftharpoons$ 30

If the node is already known:

Time complexity = $O(1)$

If the node must first be searched:

Overall complexity = $O(n)$.

Q14. Compare singly and doubly linked lists.

Answer:

Feature	Singly Linked List	Doubly Linked List
Links/node	One	Two
Direction	Forward	Forward and backward
Memory	Less	More
Backward traversal	Not directly possible	Possible
Node deletion	Requires predecessor	Easier if node known
Complexity of traversal	$O(n)$	$O(n)$

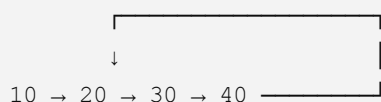
Thus, a doubly linked list provides greater flexibility at the cost of additional memory.

Q15. Explain circular singly linked lists with an example.

Answer:

In a circular singly linked list, the last node points back to the first node.

Example:



There is no `NULL` link at the end.

A traversal can be performed as:

```
current = head

do
    process(current->data)
    current = current->next
while current != head
```

Complexity

Traversal of n nodes:

$O(n)$

Q16. Explain insertion at the beginning of a circular linked list using a tail pointer.

Answer:

Suppose:

```
tail
↓
30 → 10 → 20
    ↑   |
    └───┘
```

The first node is:

```
tail->next
```

To insert `5`:

```
newNode->next = tail->next
tail->next = newNode
```

Now:

```
30 → 5 → 10 → 20
    ↑   |
    └───┘
```

The tail pointer does not need to change.

Complexity

$O(1)$

Q17. Explain insertion at the end of a circular linked list using a tail pointer.

Answer:

Suppose:

```
10 → 20 → 30
↑         ↓
└────────┘
```

tail points to 30 .

To insert 40 :

```
newNode->next = tail->next
tail->next = newNode
tail = newNode
```

Result:

```
10 → 20 → 30 → 40
↑         ↓
└────────┘
```

Complexity

$O(1)$ because the tail node is directly available.

Q18. Explain deletion from the beginning of a circular linked list.

Answer:

With a tail pointer:

```
head = tail->next
```

Algorithm

```
DELETE_BEGIN(tail)

1. If tail == NULL:
    UNDERFLOW
2. head = tail->next
3. If head == tail:
    tail = NULL
4. Else:
    tail->next = head->next
5. Delete head
```

Complexity

O(1)

This handles both the single-node and multiple-node cases.

Q19. Compare linear, doubly and circular linked lists.

Answer:

Feature	Singly	Doubly	Circular
Links	One	Two	One or two
Forward traversal	Yes	Yes	Yes
Backward traversal	No	Yes	Depends on type
Last node	NULL	NULL	Points to first
Extra memory	Low	Higher	Depends
Common use	General lists	Navigation/editing	Round-robin scheduling

Each type is selected according to the requirements of the application.

Q20. Explain the time complexity of common linked-list operations.

Answer:

For a singly linked list:

Operation	Time Complexity
Access first node	$O(1)$
Access kth node	$O(n)$
Traversal	$O(n)$
Search	$O(n)$
Insert at beginning	$O(1)$
Insert after known node	$O(1)$
Insert at end without tail	$O(n)$
Insert at end with tail	$O(1)$
Delete first node	$O(1)$
Delete known node with predecessor	$O(1)$
Delete last node	$O(n)$

The key principle is that **finding a node generally requires $O(n)$, while changing links after the required position is known can often be done in $O(1)$.**